

# Routing3D · 기존설계 패턴 학습 기반

## 자동라우팅 — 개발계획

---

양 끝 스텝(장비 출발 / 덕트·레터럴 진입) 학습 → 패턴 구축 · 추론 · 적용 · 단위 mm · 작성  
2026-06-01

### 1. 개요 · 배경

Routing3D 엔진은 PostgreSQL(AUTOROUTINGV7)의 BIM 장애물·메인장비 PoC 를 입력으로 충돌 없는 직교 배관 경로를 자동 산출한다. 직전 작업에서 '덕트 상부 PoC 로 가야 할 배관이 덕트를 관통'하는 문제를 (a) 설비·덕트·레터럴을 항상 솔리드 장애물로 처리, (b) PoC 를 솔리드 표면 바로 바깥으로 투영(LiftPocToSurface), (c) 장비 출발부를 아래로 내리기(DropStartBelowEquipment), (d) 가중  $A*(w\_heur)$  로 우회 탐색 — 의 네 가지 기하 규칙으로 해결했다.

그러나 이 네 규칙은 '사람이 보면 당연한' 출발·종단 형상을 손으로 코딩한 고정 규칙이다. 실제 플랜트에서는 장비 종류·유틸리티마다 '장비에서 어느 면으로 나와 어느 방향으로 꺾여 랙에 올라타는지', '덕트·레터럴에는 어느 축으로 접근해 상부로 진입하는지'가 정형적 패턴을 이룬다. 이 패턴을 기존 설계배관(TB\_ROUTE\_PATH)에서 학습해 자동라우팅에 활용하면, 고정 규칙이 놓치는 케이스를 일반화하고 결과를 '기존 설계 손맛'에 가깝게 만들 수 있다.

본 문서는 그 학습 기반 라우팅의 ① 프로세스, ② 알고리즘, ③ 학습을 통한 패턴 구축, ④ 패턴 추론, ⑤ 단계별 개발계획을 정의한다. 엔진에 이미 존재하는 corridor(회랑) 셀 집합 +  $w\_corridor$ (소프트 바이어스) 인프라(cost.hpp)와 기존배관 폴리라인 로더를 출발점으로 한다.

### 2. 문제 정의 — 왜 패턴 학습인가

- 고정 규칙의 한계: LiftPocToSurface 는 '가장 가까운 면 + 1/2셀'로만 투영한다. 그러나 실제 설계는 '덕트는 항상 위로 진입', '장비는 측면으로 나와 위로 상승' 처럼 면

선택·방향에 도메인 관례가 있어, 최근접 면이 관례와 다르면 어색하거나 비효율적 경로가 된다.

- 출발·종단 구간(스텝)이 경로 품질을 지배한다: 중간 구간은 빈 공간 직선이 대부분이고, 꺾임·관통·실패는 거의 양 끝 혼잡부에서 발생한다. 즉 '양 끝 스텝'만 잘 처리하면 품질 대부분이 결정된다.
- 기존 설계가 정답 셋이다: TB\_ROUTE\_PATH 에 수천 개의 사람이 그린 경로가 있고, 각 경로는 출발 PoC·종단 PoC 좌표와 폴리라인을 갖는다. 이는 (장비·유틸리티)→(출발 스텝 형상), (덕트·유틸리티)→(진입 스텝 형상)의 지도학습 라벨로 그대로 쓸 수 있다.
- 검증 가능성: 학습된 패턴으로 라우팅한 결과를 동일 PoC 의 기존 경로와 1:1 비교(꺾임 수·종단 정합·간섭/여유)할 수 있어, 개선을 정량 입증할 수 있다(뷰어의 '기존설계 비교' 기능과 연계).

### 3. 핵심 아이디어 — 경로를 '스텝 + 중간'으로 분해

하나의 배관 경로를 세 부분으로 본다. ① 출발 스텝(장비 PoC 에서 나와 주배관랙/자유공간에 도달하기까지), ② 중간(자유공간 직교 탐색), ③ 종단 스텝(자유공간에서 덕트·레터럴 진입 PoC 까지). 학습 대상은 ①과 ③의 형상이며, ②는 기존 A\* 그대로 둔다.

|                                                                 |          |                 |
|-----------------------------------------------------------------|----------|-----------------|
| [장비 PoC] —출발 스텝→ [랙/자유공간] — 중간(A*) — [자유공간] —종단 스텝→ [덕트 상부 PoC] |          |                 |
| ↳ 학습: (장비유형·유틸)                                                 | ↳ 그대로 A* | ↳ 학습: (덕트유형·유틸) |
| ·나가는 면( $\pm x/\pm y/\pm z$ )                                   |          | ·접근 축           |
| ·1 차 방향·상승 높이                                                   |          | ·진입 면(주로 +z 상부) |
| ·랙 진입 z 레벨                                                      |          | ·여유(오프셋) 거리     |

스텝은 '국소 형상'이므로 장비/덕트의 AABB 로컬 프레임으로 정규화하면 위치·치수가 달라도 동일 패턴으로 모인다. 학습은 이 정규화 공간에서 '대표 스텝(템플릿)'을 만드는 일이고, 추론은 새 PoC 에 대해 (키로 조회한) 템플릿을 월드 좌표로 역변환해 적용하는 일이다.

## 4. 데이터 소스

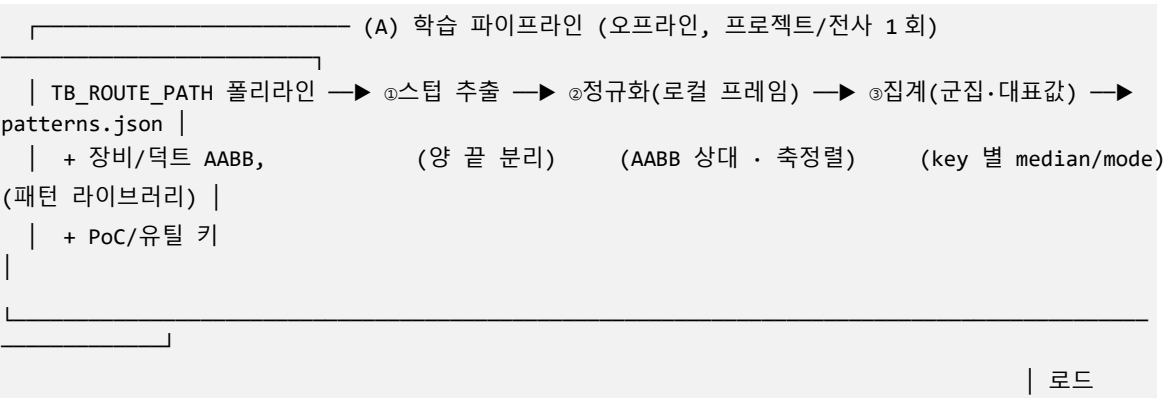
모두 동일 BIM 월드 프레임(mm)이라 좌표를 그대로 짝지을 수 있다. 격자  $origin=lo$ ,  $shape=ceil((hi-lo)/cell)$ .

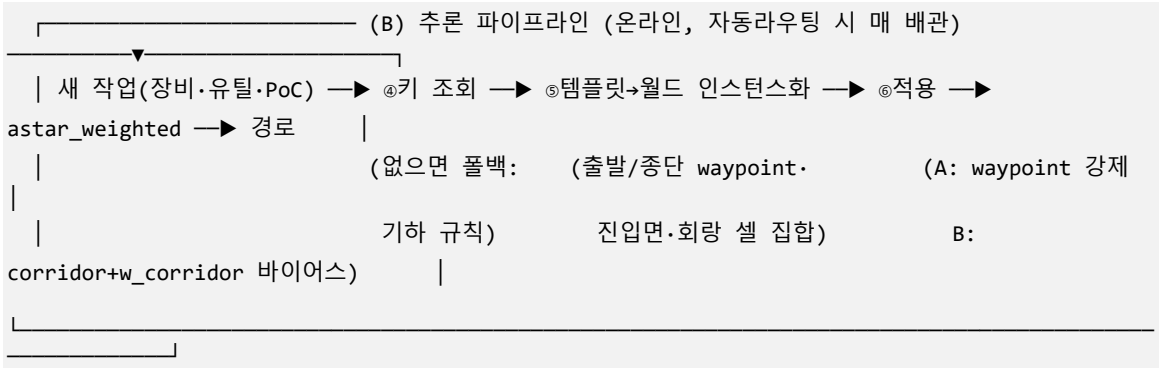
| 소스                                 | 내용                                                                 | 학습에서의 역할                                             |
|------------------------------------|--------------------------------------------------------------------|------------------------------------------------------|
| TB_ROUTE_PATH<br>(+SEGMENT/DETAIL) | 사람이 설계한 배관 폴리라인 + SOURCE_POS/TARGET_POS(PoC) + UTILITY/GROUP + 호칭경 | 지도학습 라벨(정답 경로). 양 끝 스텝 추출 원천. 이미 ExistingPipe 로 로드됨. |
| TB_BIM_EQUIPMENT<br>(+POC_LIST)    | 메인장비 AABB + PoC(pocPosition, utility, utilityGroup, endPocs)       | 출발 스텝의 로컬 프레임(장비 AABB) + 키(장비·유틸). 자동작업 생성원.         |
| TB_DUCT_LATERAL                    | 덕트-레터럴 AABB + UTILITY + CATEGORY                                   | 종단 스텝의 로컬 프레임(덕트 AABB) + 키(덕트유형·유틸). 진입면 학습.         |
| TB_BIM_OBSTACLES                   | 장애물 AABB                                                           | 스텝/경로의 간섭·여유(클리어런스) 평가, 추론 적용 시 충돌 검사.               |

주의: TB\_ROUTE\_PATH 폴리라인은 종단 PoC 너머로 연장된 트렁크/매니폴드 row 를 포함할 수 있어, 로더가 이미 SourcePos/TargetPos 안쪽으로 절단(TrimToBoundary)한다 — 학습도 절단된 폴리라인을 쓴다.

## 5. 전체 프로세스 아키텍처

두 파이프라인으로 나눈다. (A) 오프라인 학습 파이프라인: 기존배관 → 스텝 추출 → 정규화 → 집계 → 패턴 라이브러리(JSON). (B) 온라인 추론 파이프라인: 새 작업 → 키 조회 → 템플릿 인스턴스화 → (waypoint 강제 / 회랑 바이어스) → A\* → 결과. 학습은 무겁지만 1 회, 추론은 가볍게 매 배관.





폴백 원칙: 키에 해당하는 학습 패턴이 없거나 신뢰도가 낮으면 현재의 기하 규칙(Lift/Drop)으로 안전하게 되돌아간다 — 학습은 '있으면 더 좋게', 없으면 현행 유지(무해).

## 6. 패턴 표현 (자료구조)

패턴의 최소 단위는 StubTemplate(스텝 템플릿)이다. 장비/덕트 AABB 의 로컬 프레임에서 '어느 면으로 나가/들어가, 어떤 방향 시퀀스로, 얼마나 상승/오프셋하는지'를 기술한다.

| 필드        | 타입                                                 | 설명                                                                     |
|-----------|----------------------------------------------------|------------------------------------------------------------------------|
| key       | (anchor_kind, anchor_type, utility_group, utility) | 조회 키.<br>anchor_kind=EQUIP DUCT,<br>anchor_type=장비/덕트<br>분류(또는 크기 버킷). |
| face      | enum {+x,-x,+y,-y,+z,-z}                           | PoC 가 빠져나오/들어가는 앵커<br>AABB 면(로컬). 종단은 대개<br>+z(상부).                    |
| dir_seq   | list[axis-sign]                                    | 스텝의 정규화된 방향 시퀀스(예:<br>[+z, +x] = 위로 튕 뒤 +x 로).<br>꺾임 수=len-1.          |
| rise_mm   | float                                              | 면에서 첫 꺾임/랙까지의<br>수직(또는 법선) 이동 거리(mm).<br>분포의 중앙값.                      |
| offset_mm | float                                              | 면 법선 방향 진입/이탈<br>여유(LiftPocToSurface 의 ½셀<br>일반화).                     |
| rack_z_mm | float   null                                       | 출발 스텝이 합류하는 주배관랙<br>z 레벨(rack_levels 학습값).<br>없으면 null.                |
| n         | int                                                | 이 템플릿을 지지하는                                                            |

|        |       |                                      |
|--------|-------|--------------------------------------|
|        |       | 표본(기준배관) 수, 신뢰도.                     |
| spread | float | 표본 분산(방향 일치율·rise 표준편차). 낮을수록 강한 패턴. |

patterns.json = StubTemplate 의 배열. 출발 템플릿(EQUIP)과 종단 템플릿(DUCT)을 같은 스키마로 담는다. 추가로 rack 레벨 집계(z 히스토그램 상위 모드)를 전역 섹션에 둔다(7.3.8.3 참조).

## 7. 알고리즘 — 학습(패턴 구축)

### 7.1 스텝 추출 (Stub Extraction)

각 기준배관 폴리라인 P=[p0..pm] 에 대해 양 끝에서 스텝을 잘라낸다. 출발 스텝은 SourcePos 쪽, 종단 스텝은 TargetPos 쪽. 스텝 경계는 '앵커(장비/덕트) AABB 를 벗어나 자유공간 직선으로 안정될 때까지' 로 정의한다.

1. PoC 쪽 끝점을 앵커 AABB(장비 또는 덕트)에 매칭: PoC 좌표를 포함/최근접하는 AABB 를 찾는다.
2. 폴리라인을 PoC 에서부터 따라가며, 누적 길이가 L\_stub(예: max(3 셀, 장비 대각의 일부)) 이내거나 방향이 2 회 이상 바뀌기 전까지를 스텝 구간으로 절취. (자유공간 긴 직선이 시작되면 종료.)
3. 스텝 구간을 직교 축으로 스냅(거의 축정렬이므로 작은 사선은 가장 가까운 축으로) → 방향 시퀀스 dir\_seq 화.
4. 면(face)=PoC 가 앵커 AABB 의 어느 면에 가장 가까운지로 결정. rise\_mm=면 법선 방향 첫 구간 길이.

### 7.2 정규화 (Local-Frame Normalization)

스텝을 앵커 AABB 로컬 프레임으로 옮긴다. 원점=PoC, 축=월드축(직교 라우팅이므로 회전 불필요), 스케일=mm 유지(또는 셀 단위). 이렇게 하면 장비 위치·크기가 달라도 '면+방향시퀀스+rise'가 동일 패턴으로 정렬된다. 좌우 대칭( $\pm x$ ,  $\pm y$ )은 필요 시 정준화(canonicalize)해 같은 군으로 묶는다.

```
normalize(stub, anchor):
    face    = nearest_face(stub.poc, anchor.aabb)          # +z 등
    local   = [pt - stub.poc for pt in stub.pts]          # PoC 기준 상대
    dirs    = axis_snap(diff(local))                      # [+z,+x,...]
```

```

rise = length_along(local, normal(face))          # 면 법선 첫 이동
return StubSample(face, dirs, rise, offset, util_key)

```

### 7.3 집계 (Aggregation → 대표 템플릿)

같은 key((anchor\_kind, anchor\_type, utility\_group, utility))로 StubSample 들을 모아 대표 StubTemplate 을 만든다. 방향성 데이터라 '평균'이 아니라 '최빈/중앙값'을 쓴다.

- face = 최빈 면(mode). 동물이면 표본 다수 + 도메인 우선순위(종단=+z 선호).
- dir\_seq = 최빈 방향 시퀀스(시퀀스 자체를 범주로). 길이가 다르면 짧은 쪽으로 정렬 후 다수결.
- rise\_mm / offset\_mm = 중앙값(median, 이상치에 강함). spread=IQR 또는 표준편차로 신뢰도화.
- rack\_z\_mm = 출발 스텝이 합류하는 z 의 전역 히스토그램 상위 모드(주배관랙 레벨) → rack\_levels 로 환원.
- n < n\_min(예: 5) 인 key 는 상위 키로 폴백(utility 무시하고 group, 또는 anchor\_type 만).

산출물 patterns.json 은 사람이 검수 가능한 평문(면·방향·치수·표본수)이라, 학습 결과를 도메인 전문가가 확인·보정할 수 있다(블랙박스 아님).

## 8. 알고리즘 — 추론 · 적용

### 8.1 템플릿 조회 · 월드 인스턴스화

새 작업(장비·유틸·PoC)에 대해 key 로 출발/종단 StubTemplate 을 조회하고, 해당 장비/덕트 AABB 와 PoC 로 월드 좌표 waypoint 열을 만든다(정규화의 역연산). 키 미스면 폴백.

```

infer(task):
    st = lookup(EQUIP, eq_type, util_grp, util)  # 출발
    en = lookup(DUCT, duct_type, util_grp, util) # 종단
    start_wps = instantiate(st, eq.aabb, task.start_poc) # [PoC, PoC+rise·n(face),
+dir_seq...]
    end_wps = instantiate(en, duct.aabb, task.end_poc) # 덕트 상부 진입 보장
    return start_wps, end_wps # 미스 → Lift/Drop 기하 폴백

```

### 8.2 적용 3 수준 (A · B · C)

| 수준 | 방식 | 엔진 연계 | 장점 / 한계 |
|----|----|-------|---------|
|----|----|-------|---------|

|              |                                                         |                                                                                 |                                                  |
|--------------|---------------------------------------------------------|---------------------------------------------------------------------------------|--------------------------------------------------|
| A · 하드 스텝    | 출발/종단 waypoint 를 강제 통과점으로 두고 그 사이만 A* 로 연결(구간 연쇄 탐색).   | 신규: 구간별 astar_weighted 호출 + waypoint 고정. LiftPocToSurface 의 학습판.                | 패턴을 정확히 재현(관통·면선택 문제 직결 해결). 단, 강제점이 막히면 폴백 필요.  |
| B · 소프트 바이어스 | 학습된 스텝/랙을 회랑(corridor) 셀 집합으로 만들어 그 밖을 w_corridor 로 가산. | 기존 인프라 재사용: cost.hpp 의 corridor 집합 + w_corridor + rack_levels (이미 구현·C ABI 노출). | 막힘에 강함(강제 아님, 비용만 유도). A* 최적성 유지. 효과는 가중치 튜닝 의존. |
| C · 랙/번들 학습  | 유틸리티 그룹별 주배관랙 z 레벨·간선 회랑을 학습해 다수 배관을 같은 랙에 번들링.         | rack_levels + 그룹 공유 corridor 집합. route_corridor_multi 와 결합.                     | 전사적 정연함(랙 정렬)·길이 ↓. 학습·집계 비용 큼. L3 후반 목표.        |

권장 도입 순서: B(소프트) → A(하드) → C(랙). B 는 이미 있는 corridor/w\_corridor 인프라에 '학습된 회랑 셀'만 채워 넣으면 되므로 리스크가 가장 낮고, 관통 같은 강한 케이스는 A 로 보강한다. A 의 waypoint 가 폴백할 때 B 의 바이어스가 안전망이 된다(상호 보완).

### 8.3 B 수준 — corridor 인스턴스화 상세 (이미 있는 인프라 활용)

cost.hpp 는 corridor(=occ.lin 인덱스 집합)에 속하지 않는 셀마다 w\_corridor(mm)를 가산하고, rack\_levels(z) 는 회랑으로 간주해 면제한다. 추론은 다음을 채운다.

- 출발/종단 스텝 waypoint 주변( $\pm$ corridor\_radius 셀)을 회랑 셀로 추가 → 학습된 진입 형상을 따라가도록 유도.
- 학습된 rack\_z\_mm → rack\_levels(z 셀 인덱스)로 변환 → 주배관랙 높이로 자연 유도.
- 회랑 밖 가산(w\_corridor)이라 '강제'가 아니므로, 회랑이 막히면 A\* 가 비용을 더 내고 우회(안전).
- C ABI: R3dParams.w\_corridor/corridor\_radius + (신규) 회랑 셀 주입 API 또는 route\_corridor\_multi 경유.

## 9. 단계별 개발계획 (L1 → L2 → L3)

| 단계             | 목표                    | 산출물                                   | 검증                               |
|----------------|-----------------------|---------------------------------------|----------------------------------|
| L1 · 추출/학습 PoC | 기존배관에서 스텝 추출·정규화·집계 → | Python: routing3d_py/pattern_learn.py | 표본수·면 분포·dir_seq 최빈·rise 중앙값을 표로 |

|                   |                                                                       |                                                                               |                                                             |
|-------------------|-----------------------------------------------------------------------|-------------------------------------------------------------------------------|-------------------------------------------------------------|
|                   | patterns.json<br>생성·검수                                                | (추출·정규화·집계),<br>out/patterns_<proj>.json,<br>통계 리포트                           | 출력. 사람이 '말이<br>되는지' 검수.                                     |
| L2 · B 적용(소프트)    | patterns.json → 회랑<br>셀/rack_levels 로<br>변환해 자동라우팅에<br>주입(w_corridor) | C# 뷰어 BuildEngineForRows<br>에 패턴 로더 + 회랑 주입, C<br>ABI 회랑 셀 주입(필요 시),<br>토글 UI | 관통 0 유지 + 꺾임<br>수↓ + 기존설계<br>중단정합↑를 '기존설계<br>비교' 패널로 정량 확인. |
| L3 · A/C 적용(하드·랙) | waypoint 강제<br>구간연쇄 + 유틸그룹<br>랙 번들링                                   | 구간연쇄 라우팅(엔진/래퍼),<br>rack 학습,<br>route_corridor_multi 결합                       | project6/CMP 전체에서<br>성공률·충격이·꺾임·관통<br>대비표(학습 OFF vs<br>ON). |

## 9.1 L1 상세 — 학습 PoC (가장 먼저)

- 신규 파일 python\_experiments/routing3d\_py/pattern\_learn.py — 모듈 상단에 실행명령어 블록(프로젝트 규약).
- 입력: obstacle\_db/scene 로 로드한 ExistingPipe(폴리라인+PoC) + 장비/덕트 AAB.
- 함수: extract\_stub(), normalize\_stub(), aggregate(samples)->templates, dump\_patterns(json).
- CLI: python -m routing3d\_py.pattern\_learn --project 6 --out out/patterns\_p6.json --report.
- 리포트(검수용): key 별 n·face 분포·dir\_seq Top3·rise(med/IQR)·rack z 모드 표.

## 9.2 L2 상세 — 소프트 바이어스 적용

- C# Model 신규 PatternLibrary.cs — patterns.json 로드 + key 조회 + 월드 회랑 셀 생성.
- BuildEngineForRows: 작업별 출발/종단 템플릿 조회 → 회랑 셀 모음 → 엔진 주입 + w\_corridor>0, rack\_levels 설정.
- C ABI: 회랑 셀 집합 주입 경로 확정 — 기존 r3d\_route\_corridor\_multi 활용 또는 r3d\_set\_corridor\_cells 신설(POD int 배열).
- UI: '기존설계 패턴 활용' 토글(기본 ON, 미스면 자동 폴백). 학습 OFF=현행 동작(회귀 안전).
- 엔진/Python 골든 불변: w\_corridor 기본 0 경로는 바이트 동일(기존 ctest/pytest 영향 없음).

### 9.3 L3 상세 — 하드 스텝 · 랙 번들

- 구간연쇄: start\_wps→...→end\_wps 를 순서대로 astar\_weighted 로 잇고 실패 구간만 폴백. set\_task\_endpoints 재사용.
- 랙 학습: 유틸그룹별 z 히스토그램 → 대표 랙 레벨 → 그룹 공유 회랑으로 다배관 번들링.
- 평가: 학습 OFF/ON 대비표(성공률·총길이·평균 꺾임·관통 셀·소요 ms)를 회귀 리포트에 추가.

## 10. 평가 지표

| 지표        | 정의                                       | 목표                         |
|-----------|------------------------------------------|----------------------------|
| 관통 셀 수    | 경로 샘플점이<br>솔리드(설비/덕트/레터럴) AABB<br>내부인 개수 | 0 유지(현행 보존)                |
| 종단 정합     | 라우팅 종단점과 PoC/덕트 상부<br>진입면 거리(mm)         | 기존설계 수준 이하                 |
| 평균 꺾임 수   | 경로당 방향전환 횟수(엘보)                          | 학습 OFF 대비 감소               |
| 기존설계 유사도  | 동일 PoC 기존경로와 스텝<br>방향·면 일치율(%)           | 상승(정성·정량)                  |
| 성공률 / 총길이 | 충돌 없이 라우팅된 비율 / 길이<br>합                  | 성공률 유지·상승, 길이 과증가<br>없음    |
| 소요 시간     | 배관당/전체 ms                                | 현행 대비 과도하지 않게(가중 A*<br>수준) |

측정 도구는 이미 있다: 뷰어의 '기존설계 비교'(꺾임/정합/간섭/여유) + 회귀 리포트  
생성기(\_gen\_regression\_report.py)에 학습 OFF/ON 컬럼을 추가한다.

## 11. 리스크 · 완화

| 리스크             | 영향         | 완화                                             |
|-----------------|------------|------------------------------------------------|
| 표본 부족(희귀 유틸)    | 신뢰 낮은 템플릿  | n_min 미만은 상위 키 폴백 →<br>최종 기하 규칙 폴백(무해).        |
| 과적합(특정 프로젝트 관례) | 타 프로젝트 부적합 | 프로젝트별 학습 우선 + 전사<br>평균은 별도. spread 로 신뢰<br>가중. |
| 하드 waypoint 막힘  | 구간 실패      | 구간연쇄 실패 시 B<br>바이어스/기하 폴백으로 자동                 |

|            |           |                                                  |
|------------|-----------|--------------------------------------------------|
|            |           | 강등.                                              |
| 골든/회귀 깨짐   | 기존 검증 무효화 | 학습 OFF=현행 바이트 동일 보장. w_corridor 기본 0. 신규 경로만 분기. |
| 좌표 프레임 불일치 | 스텝 오정렬    | 라우트=BIM 동일 mm 프레임 확인(로더 TrimToBoundary 와 동일 가정). |
| 성능 저하      | 대형 격자 느려짐 | 회랑은 탐색을 오히려 좁혀 가속 가능. 학습은 오프라인 1 회.              |

## 12. 일정 · 마일스톤 (제안)

| 마일스톤    | 범위                                                        | 완료 기준                                |
|---------|-----------------------------------------------------------|--------------------------------------|
| M1 (L1) | pattern_learn.py<br>추출·정규화·집계 + patterns.json<br>+ 검수 리포트 | project6 패턴이 도메인 상식과 일치(전문가 검수 OK)   |
| M2 (L2) | PatternLibrary + 회랑 주입 +<br>w_corridor 적용 + 토글            | CMP 덕트 케이스 관통 0 유지 +<br>종단정합 ↑ 정량 확인 |
| M3 (L3) | 하드 스텝 구간연쇄 + 랙 번들 +<br>회귀 OFF/ON 대비표                      | 성공률·꺾임·길이 종합 개선 입증,<br>회귀 리포트 갱신     |

각 마일스톤은 독립적으로 가치를 낸다(L1 만으로도 '기존설계 비교' 분석이 풍부해지고, L2 만으로도 라우팅 품질이 오른다). 따라서 L1→L2→L3 순으로 점진 도입하고, 단계마다 회귀(golden/capi/implicit + pytest + --selftest)로 현행 보존을 확인한다.

## 13. 구현 현황 · 실측 (2026-06-02)

본 계획의 L1(학습 파이프라인 + pgvector 저장소)·L2a(학습 면 PoC 투영)와 접근불가 PoC 전처리를 구현·검증했다. 엔진/C ABI 변경 없이 Python 학습 + C# 뷰어 적용만으로 완성했고, DB 에 설치된 pgvector 를 스텝 표본의 벡터 저장·검색에 사용한다.

### 13.1 산출물

| 계층    | 파일                               | 역할                                                                                 |
|-------|----------------------------------|------------------------------------------------------------------------------------|
| L1·DB | db/schema/route_stub_pattern.sql | 벡터 스키마(feat<br>vector(24)·dir_unit<br>vector(3)·HNSW) + 집계뷰<br>route_stub_template |
| L1·Py | routing3d_py/route_db.py         | 기존배관(TB_ROUTE_PATH 3-                                                              |

|        |                               |                                                                      |
|--------|-------------------------------|----------------------------------------------------------------------|
|        |                               | join):장비·덕트 로더(C#→Python 포팅)                                         |
| L1·Py  | routing3d_py/pattern_learn.py | 스텝 추출→로컬프레임<br>정규화→특징벡터/방향벡터                                         |
| L1·Py  | routing3d_py/pattern_db.py    | 스키마 적용·벡터 적재·ANN<br>검색(nearest_stubs: 범주 pre-<br>filter+feat<->q)    |
| L2a·C# | Model/PatternStore.cs         | route_stub_template<br>조회(키→face/rise,<br>(kind,group)→(kind) 폴백)    |
| L2a·C# | ViewModels/SceneViewModel.cs  | LiftPocToSurface 학습면 +<br>접근불가 SnapPocToFreeCell +<br>UsePatterns 토글 |

### 13.2 학습 결과 — 도메인 규칙을 데이터로 입증

project6 → 405 스텝 표본 / 38 키 적재. 학습된 대표 면이 도메인 상식과 정확히 일치한다:  
출발(EQUIP)=-z(장비에서 아래로), 종단(DUCT)=+z(덕트 상부로 진입). 즉 기존 기하  
규칙(DropStartBelowEquipment·LiftPocToSurface 상부 우선)이 사람 설계 데이터로  
검증되었고, 패턴은 이를 키별로 일반화해 최근접-면 휴리스틱이 틀리는 코너 케이스를  
바로잡는다.

### 13.3 실측 A/B (project6, cell=100, 208 작업)

| 구성                 | 성공        | 시간     | 총길이(mm)   |
|--------------------|-----------|--------|-----------|
| 무패턴·무스냅 (기준)       | 185 / 208 | 23.6 s | 1,474,000 |
| + 학습면 투영 (L2a)     | 187 / 208 | 6.4 s  | 1,470,000 |
| + 접근불가 스냅<br>(전처리) | 194 / 208 | 6.7 s  | 1,442,800 |

누적 효과: +9 성공 · 3.5× 빠름 · 총길이 감소 · 회귀 0. L2a 의 속도 향상은 잘못된 면으로  
나가 수백만 노드를 헛탐색하던 배관이 학습 면 투영으로 사라진 효과다. 전처리는 솔리드에  
파묻힌 PoC 7 건을 최근접 자유셀로 복구했다. 남은 14 실패는 expanded>0(끝점은  
풀렸으나 경로 없음 = 혼잡/막힘)으로 rip-up/CBS 영역이며 패턴 범위 밖이다.

### 13.4 L2b(회랑 소프트 바이어스) 보류 사유

- cost.hpp 의 corridor 셋이 unordered\_set<int> + static\_cast<int>(occ.lin) 라  
ImplicitOccupancy(64 비트 lin) 대형 격자에서 키 절단 위험 → 안전 적용에  
64 비트화(cost.hpp/corridor.hpp/ctest 골든) 필요.

- C ABI r3d\_set\_corridor\_cells 신설 + DLL 재빌드 + 골든 회귀 위험(네이티브 수술).
- 개선 대상이 '이미 성공하는 경로의 모양(번들링·접근 형상)'이고 실패 감소가 아님 — L2a+ 전처리로 핵심 가치를 이미 확보 → ROI 낮아 보류. 필요해지면 64 비트화부터 착수.

## 14. 결론

현 엔진의 기하 규칙(Lift/Drop/가중 A\*)은 '학습 패턴의 하드코딩 특수해'다. 기존 설계배관을 (장비·유틸)·(덕트·유틸) 키로 학습해 양 끝 스텝을 일반화하면, 관통·면선택 같은 약점을 도메인 관례로 메우고 결과를 사람 설계에 수렴시킬 수 있다. 핵심 인프라(corridor 셀 + w\_corridor + rack\_levels, 기존배관 폴리라인 로더, 기존설계 비교 분석)가 이미 갖춰져 있어, L1(학습 PoC)부터 낮은 리스크로 착수할 수 있다. 권장 착수점은 'L1 추출/집계 + L2 소프트 바이어스(B)'다.